Iris Dataset Clustering Report

1 Importing Required Libraries

In this step, the required Python libraries were imported to perform data analysis, preprocessing, visualization, clustering, and evaluation tasks. The `pandas` library was used for organizing and handling the dataset, while `matplotlib` was used to visualize the data and clustering results. The Iris dataset was loaded from the Scikit-learn library.

Furthermore, `StandardScaler` was used to normalize the features before clustering, which helps improve the performance of machine learning algorithms. Principal Component Analysis (PCA) was imported to reduce the dimensionality of the dataset and simplify visualization. The DBSCAN algorithm was used as the clustering technique to identify groups and detect noise points within the dataset. Finally, the `silhouette_score` metric was imported to evaluate the quality and effectiveness of the clustering results.

2 Loading and Displaying the Dataset

In this step, the Iris dataset was loaded using the Scikit-learn library. The dataset contains measurements of iris flowers, including sepal length, sepal width, petal length, and petal width. After loading the dataset, a Pandas DataFrame was created to organize the data into a tabular format with appropriate feature names as column headers.

Finally, the `head()` function was used to display the first five rows of the dataset in order to inspect the structure of the data and verify that it was loaded correctly.

3 Dataset Inspection and Validation

In this step, the dataset was inspected to ensure its quality and structure before applying preprocessing and clustering techniques. First, missing values were checked using the `isnull().sum()` function to verify that the dataset did not contain incomplete records.

Next, the shape of the dataset was displayed to identify the number of rows and columns contained in the data. The column names were also printed to confirm the available features in the dataset. Finally, the data types of all columns were examined to ensure that the features were stored in a suitable numerical format for machine learning and clustering operations.

4 Feature Scaling

In this step, feature scaling was applied to the dataset using the `StandardScaler` technique. Standardization is an important preprocessing step in machine learning because it transforms all features to have a mean of zero and a standard deviation of one.

This process ensures that all variables contribute equally to the clustering algorithm and prevents features with larger numerical ranges from dominating the analysis. The scaled dataset was then stored in a new variable to be used in the following clustering and dimensionality reduction steps.

5 Dimensionality Reduction Using PCA

In this step, Principal Component Analysis (PCA) was applied to reduce the dimensionality of the dataset from multiple features to two principal components. This reduction simplifies the dataset while preserving most of the important information and variance contained in the original data.

The transformed data was stored in a new variable for easier visualization and clustering analysis. In addition, the explained variance ratio was calculated and displayed to measure how much information from the original dataset was retained after dimensionality reduction.

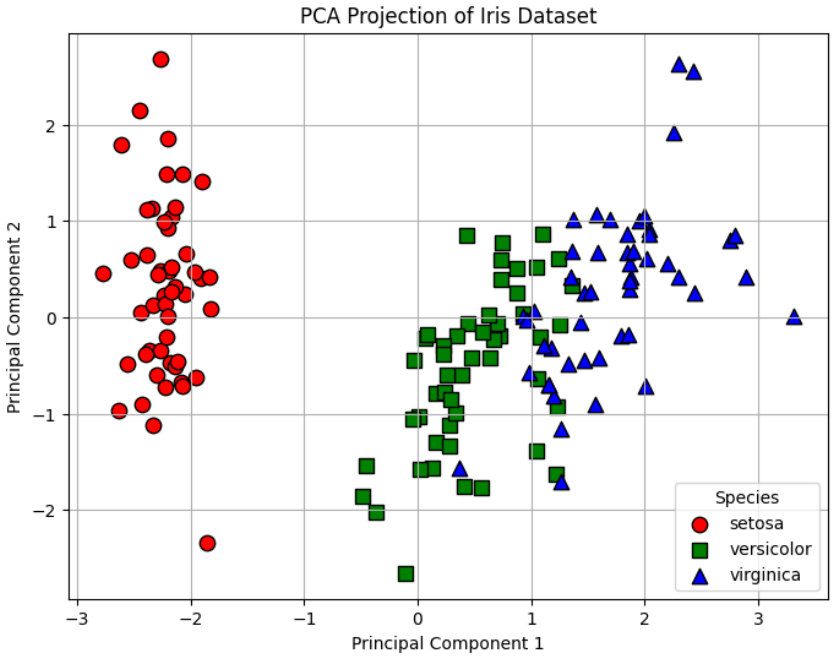

Figure 1: PCA dimensionality reduction results

6 DBSCAN Clustering on the Original Dataset

In this step, the DBSCAN clustering algorithm was applied to the standardized dataset in order to identify clusters and detect noise points. The algorithm was configured using specific values for the `eps` and `min_samples` parameters, which control the neighborhood distance and the minimum number of points required to form a cluster.

After fitting the model to the scaled data, cluster labels were generated for all data points. The clustering results were then visualized using a scatter plot, where each cluster was represented with different markers and colors. Noise points identified by DBSCAN

were displayed separately using black cross markers to distinguish them from the clustered observations.

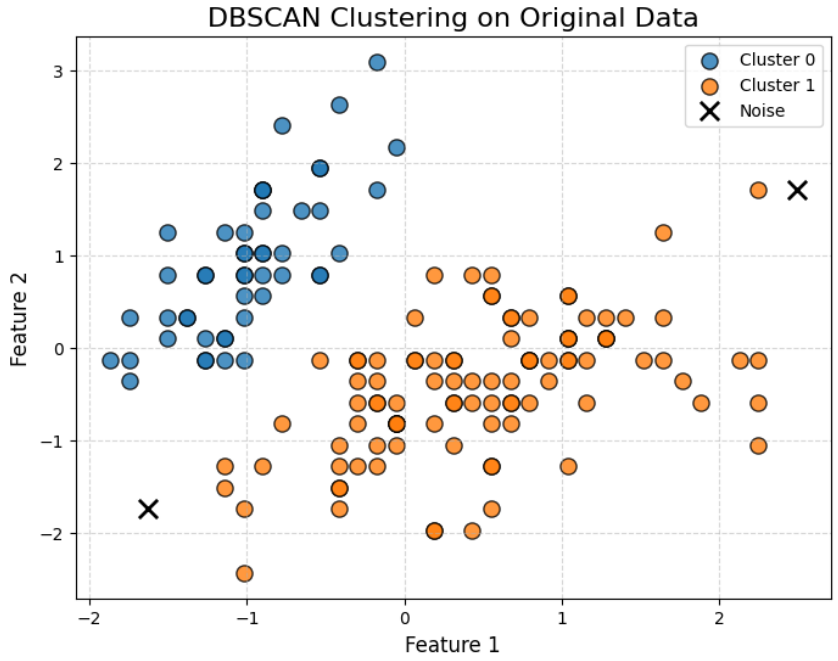

Figure 2: DBSCAN clustering results on the original dataset

7 DBSCAN Clustering After PCA

In this step, the DBSCAN clustering algorithm was applied again after reducing the dataset dimensionality using Principal Component Analysis (PCA). The clustering process was performed on the two principal components generated by PCA, which simplified the dataset while preserving most of its important information.

The DBSCAN model was configured using selected values for the `eps` and `min_samples` parameters. After fitting the algorithm to the PCA-transformed data, cluster labels were generated for all observations.

The clustering results were visualized using a scatter plot based on the two principal components. Different clusters were displayed using separate markers, while noise points detected by DBSCAN were represented using black cross markers. This visualization made it easier to observe the separation between clusters after dimensionality reduction.

8 Parameter Tuning for DBSCAN on the Original Dataset

In this step, parameter tuning was performed to identify the optimal DBSCAN configuration for the original standardized dataset. Different combinations of the `eps` and `min_samples` parameters were tested in order to evaluate their impact on clustering performance.

For each parameter combination, the DBSCAN algorithm was applied to the scaled dataset, and the number of clusters and noise points were calculated. The silhouette

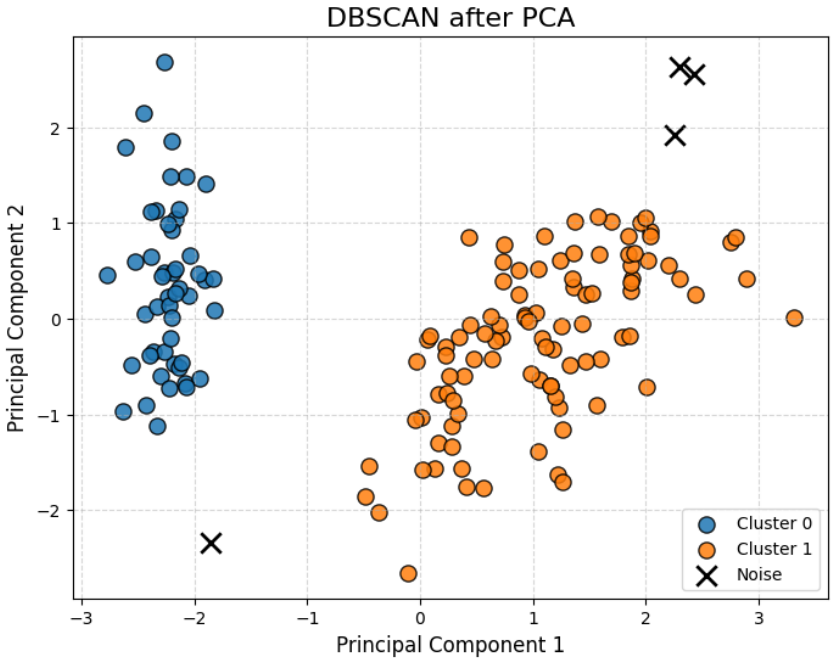

Figure 3: DBSCAN clustering results after applying PCA

score was also computed whenever more than one cluster was detected, as this metric measures the quality and separation of the resulting clusters.

The clustering results for all parameter combinations were visualized using multiple scatter plots based on the PCA-transformed data. Each subplot represented a unique parameter configuration, allowing easier comparison between clustering outcomes.

Finally, the best parameter combination was selected according to the highest silhouette score, and the optimal values for `eps` and `min_samples` were displayed.

9 Final Comparison of DBSCAN Results

In this step, a final comparison was performed between the clustering results obtained before and after parameter tuning on both the original standardized dataset and the PCA-transformed dataset. A custom function was created to apply the DBSCAN algorithm and calculate important evaluation metrics, including the number of clusters, number of noise points, and silhouette score.

Four different experimental cases were evaluated:

- Original dataset before parameter tuning
- Original dataset after parameter tuning
- PCA-transformed dataset before parameter tuning
- PCA-transformed dataset after parameter tuning

The results demonstrated that all configurations successfully detected two clusters within the dataset. However, differences appeared in the number of detected noise points and the silhouette scores, which reflect clustering quality.

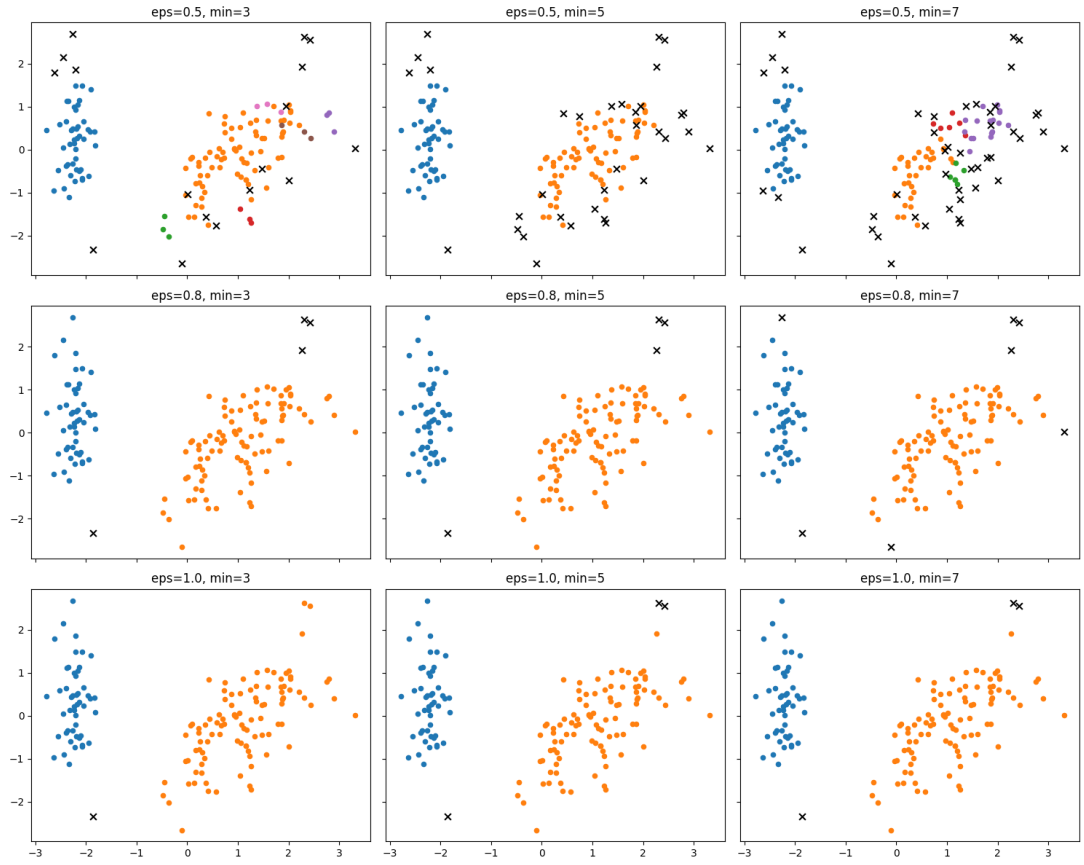

Figure 4: DBSCAN parameter tuning results on the original dataset

The highest silhouette score was achieved after applying PCA and tuning the DBSCAN parameters, with a silhouette score of 0.572201. This indicates that dimensionality reduction combined with parameter optimization improved the clustering performance and produced better cluster separation compared to the other configurations.

Data Type	Stage	eps	min_samples	Clusters	Noise Points	Silhouette Score
Original	Before Tuning	1.1	5	2	2	0.551772
Original	After Tuning	1.0	5	2	3	0.538288
PCA	Before Tuning	0.8	5	2	4	0.557590
PCA	After Tuning	1.0	7	2	3	0.572201

Table 1: Final comparison of DBSCAN clustering results